# Main Report — HW05 (Performance Testing)

**Student:** Nguyễn Tấn Phát — 23127449

August 16, 2026

## 1. Overview

This report documents the end-to-end performance testing workflow executed against the EShop SUT (System Under Test) for HW05. The entire process was driven by an **AI-first strategy**, utilizing specialized Agent Skills to design, build, execute, and analyze performance tests across four scenarios: Load, Stress, Spike, and Soak.

## 1.5. Requirement Compliance: Three Distinct Report Views

To fulfill the requirement of utilizing three different report views across the three main test scenarios (equivalent to JMeter's distinct listeners), the following k6 output mechanisms were implemented:

1. **Load Test (Read-Heavy):** `k6-reporter` **HTML Summary Report** (Visual web dashboard view).
2. **Stress Test (Auth-Heavy): JSON Raw Export** via `--out json` (Granular point-in-time metrics logging).
3. **Spike Test (Transactional): CLI stdout Text Summary** (Standard console aggregate view).

## 2. Task 1: Performance Testing Scenarios & Results

Three distinct endpoints were chosen, each representing a unique workload model, supplemented by an endurance test to determine hardware thresholds. All tests were executed against a local Docker container constrained to **2 vCPU / 1 GB RAM**.

### 2.1. Load Test (Read-Heavy)

- **Endpoint:** `GET /api/orders/:id`
- **Auth Strategy:** Per-VU Cached Token
- **Workload:** 20 VUs peak (10 minutes total duration)
- **Performance Results:**
  - **p95 Latency:** `8.60 ms` (Excellent — well below the 100 ms SLO threshold).
  - **Error Rate:** `0.00%` (All 6,414 requests passed successfully).
  - **Throughput:** `~10.06 req/s` (Matching the designed think-time models).
  - **Resource Usage:** Negligible. CPU hovered at `~1.95%` (0.039/2.0 cores) and RAM peaked at `~70 MB`.
- **Conclusion:** The system handles normal operational load for read-heavy operations flawlessly with massive remaining headroom.

### 2.2. Stress Test (Auth-Heavy)

- **Endpoint:** `POST /api/register`
- **Auth Strategy:** No Auth (Public Endpoint)
- **Workload:** Ramp up to 130 VUs over 15 minutes.

- **Report View:** JSON Raw Export (`--out json`). *Note: Due to the large file size, the `raw-output.json` is hosted on Google Drive: Download Link.*
- **Performance Results:**
  - **p95 Latency:** `205.06 ms` (Breached the 200 ms SLO threshold at high load).
  - **Error Rate:** `0.00%`
  - **Throughput:** Scaled up to `~70 VUs` before queuing effects began to dominate.
  - **Bottleneck Identified:** The test successfully overloaded the system. Due to SQLite's serialized write operations, lock contention occurred beyond 70 VUs, causing latency spikes.
- **Conclusion:** The system failed gracefully without crashing, demonstrating queue-based latency degradation (a known SQLite architectural limitation) rather than unhandled exceptions.

### 2.3. Spike Test (Transactional)

- **Endpoint:** `POST /api/cart`
- **Auth Strategy:** Per-VU Cached Token
- **Workload:** Sudden surge from 2 VUs to 100 VUs in 30 seconds.
- **Performance Results:**
  - **p95 Latency:** `5.75 ms` (Well within the 500 ms SLO threshold).
  - **Error Rate:** `0.00%`
- **Conclusion:** The cart endpoint, which performs lightweight inserts, survived the immediate 100-VU traffic spike without any significant degradation.

### 2.4. Endurance Threshold (Soak Test)

- **Endpoint:** Mixed reads (`GET /api/products` and `GET /api/orders/my-orders`)
- **Workload:** 15 VUs sustained for 15 minutes.
- **Performance Results:**
  - **Maximum stable load:** 15 VUs
  - **p95 Latency:** `11.03 ms`
  - **Memory Ceiling:** Highly stable at `~85 MB`. No memory leaks detected over the sustained period.

## 3. Task 2: AI Analysis & Misinterpretation Hunt

Following the raw data generation, AI was utilized to parse `summary.json` and `run-log.md` to generate actionable insights. During human review, several AI misinterpretations were caught:

- **Hallucinated Recommendations:** The AI repeatedly proposed using SQL-based teardown scripts and recommended database indexing optimizations that were infeasible given the assignment's black-box testing nature.
- **Misidentified Baselines:** The AI occasionally conflated container hardware constraints with host machine constraints.

All detailed analysis, misinterpretations, and human verdicts on AI recommendations are thoroughly documented in the respective `docs/results/*/report/` directories.

## 4. Task 3: Continuous Performance Testing Pipeline

A **Context-Aware Continuous Performance Testing Pipeline** was proposed to integrate performance checks into CI/CD. The pipeline leverages GitHub Actions to map `git diff` file paths to specific test suites (e.g.,

modifying `backend/routes/cart` selectively triggers the Spike and Load test).

A custom Node.js regression checker (`check-regression.js`) was implemented. It automatically fails the build if the new `p95` latency drifts by more than **20%** against the stored baseline from Task 1.

The full proposal and Mermaid flowcharts are available in `docs/results/cpt-proposal/continuous-perf-proposal.md`.

## 5. Task 4: AI Critique

Please refer to the dedicated critique document at `reports/ai-critique.md` for a comprehensive 200-300 word analysis of the AI's performance, behavioral patterns, and the lessons learned regarding AI collaboration.\n